

ĐI TÌM THỬ THÁCH 2

Các bài toán từ Kỳ thi Lập trình
Sinh viên Ba Lan 2011–2014

Bản xem trước tiếng Việt

4 bài đầu tiên của năm 2011

*Bản dịch phục vụ mục đích giáo dục và lưu hành riêng,
được thực hiện với sự cho phép của tác giả.*

BẢN THẢO • 2026

Ghi chú về bản xem trước

Bản PDF này minh họa cách trình bày dự kiến cho bản dịch hoàn chỉnh. Nội dung được ưu tiên dịch từ bản tiếng Anh. Với 33 lời giải chưa có trong bản tiếng Anh, bản tiếng Ba Lan năm 2019 (phiên bản 2.1) sẽ là nguồn chính.

Tên biến, công thức, độ phức tạp và dữ liệu vào/ra được giữ nguyên về ngữ nghĩa. Các chương trong bản xem trước vẫn cần một vòng soát kỹ thuật độc lập trước khi được xem là hoàn chỉnh.

Mục lục

Ghi chú về bản xem trước	ii
1 Hình chữ nhật cấp số cộng ★★ ★	1
1.1 Đề bài	1
1.1.1 Dữ liệu vào	2
1.1.2 Dữ liệu ra	2
1.1.3 Ví dụ	2
1.2 Lời giải	2
1.2.1 Tai nạn hàng không	3
1.2.2 Mảnh đất	3
1.2.3 Hình chữ nhật cấp số cộng	4
2 Cuộc đua đường phố Bytean ★★ ★ ★	6
2.1 Đề bài	6
2.1.1 Dữ liệu vào	7
2.1.2 Dữ liệu ra	7
2.1.3 Ví dụ	8
2.2 Lời giải	8
2.2.1 Cấu trúc của các đường trái và phải	11
2.2.2 Lời giải nhanh hơn	12
3 Liệu nó có dừng?	15
3.1 Đề bài	15

3.1.1	Dữ liệu vào	16
3.1.2	Kết quả	16
3.1.3	Ví dụ	16
3.2	Lời giải	16
3.2.1	Chứng minh tính đúng đắn	18
3.2.2	Độ phức tạp	18
3.2.3	Một phép kiểm tra bằng toán tử bit	19
3.2.4	Bài toán Collatz	19
4	Cây và đàn kiến ★★★★★	21
4.1	Đề bài	21
4.1.1	Dữ liệu vào	22
4.1.2	Dữ liệu ra	22
4.1.3	Ví dụ	22
4.2	Lời giải	23
	Thông tin nguồn	26

Hình chữ nhật cấp số cộng



Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Bộ nhớ: 128 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/ary>

Nguồn dịch: bản tiếng Anh, trang in 3–8 (trang PDF vật lý 17–22).

1.1 Đề bài

Cho một bảng gồm $n \times m$ ô vuông đơn vị. Mỗi ô chứa một số nguyên. Ta quan tâm đến các **hình chữ nhật cấp số cộng** trên bảng: đó là những hình chữ nhật ghép từ các ô vuông đơn vị sao cho các số trên mỗi hàng và mỗi cột đều tạo thành một cấp số cộng. Nhắc lại rằng trong một cấp số cộng, hiệu của hai số hạng liên tiếp luôn không đổi.

Hãy tìm hình chữ nhật cấp số cộng lớn nhất, tức là hình chứa nhiều ô vuông đơn vị nhất. Chẳng hạn, trong bảng dưới đây, hình chữ nhật cấp số cộng lớn nhất gồm chín ô:

5	3	5	7
2	4	4	4

3	5	3	1
6	3	2	4

1.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên t ($1 \leq t \leq 10\,000$), là số bộ dữ liệu. Mỗi bộ bắt đầu bằng hai số nguyên n, m ($1 \leq n, m \leq 3\,000$). Mỗi dòng trong n dòng tiếp theo chứa m số nguyên thuộc đoạn $[0, 10^9]$, lần lượt là các số trong các ô của bảng. Kích thước mỗi tệp dữ liệu vào không vượt quá 20 MB.

1.1.2 Dữ liệu ra

In ra t dòng, mỗi dòng là đáp án của một bộ dữ liệu. Đáp án là số ô vuông đơn vị nằm trong hình chữ nhật cấp số cộng lớn nhất của bảng tương ứng.

1.1.3 Ví dụ

Input	Output
2	9
4 4	6
5 3 5 7	
2 4 4 4	
3 5 3 1	
6 3 2 4	
2 3	
0 1 2	
1 2 3	

1.2 Lời giải

Để trình bày lời giải, ta dùng hai bài toán phụ quen thuộc hơn. Mỗi bài toán sau sẽ được quy về bài toán đứng ngay trước nó.

1.2.1 Tai nạn hàng không

Cho dãy n số nguyên $a_1, \dots, a_n$. Với mỗi phần tử, ta muốn tìm phần tử nhỏ hơn gần nhất nằm bên trái nó. Chính xác hơn, với mỗi $i \in \{1, \dots, n\}$, cần tìm chỉ số j lớn nhất thỏa $j < i$ và $a_j < a_i$. Để kết quả luôn tồn tại, thêm phần tử canh $a_0 = -\infty$.

Tên gọi của bài toán bắt nguồn từ một cách phát biểu khá nghiệt ngã: biết số người thiệt mạng trong các tai nạn máy bay qua từng năm; với mỗi tai nạn, cần xác định đã bao lâu rồi mới lại có một tai nạn nghiêm trọng đến thế. Trong biến thể đó, ta tìm phần tử lớn hơn gần nhất bên trái. Sau đây ta quay lại phát biểu ban đầu.

Bài toán được giải trong thời gian tuyến tính bằng cách “đi theo các mũi tên”. Duyệt dãy từ trái sang phải và gán cho mỗi phần tử một mũi tên trỏ đến phần tử nhỏ hơn gần nhất. Khi xử lý a_i , trước hết kiểm tra $a_{i-1} < a_i$. Nếu đúng, mũi tên từ a_i trỏ thẳng đến a_{i-1} . Nếu không, đi theo mũi tên của a_{i-1} đến phần tử nhỏ hơn gần nhất của nó. Tiếp tục như vậy cho đến khi gặp một phần tử nhỏ hơn a_i .

Để chứng minh thời gian chạy tuyến tính, chỉ cần chỉ ra rằng mỗi mũi tên được đi qua nhiều nhất một lần. Giả sử khi xác định mũi tên cho a_i , ta đi qua mũi tên xuất phát từ a_j , với $j < i$. Khi đó $a_j \geq a_i$; nếu không thì a_j đã là một ứng viên cho kết quả của a_i . Gọi a_k là phần tử cuối cùng tìm được cho a_i . Khi xử lý các phần tử $a_{i+1}, a_{i+2}, \dots$, ta sẽ không bao giờ gặp lại a_j trong quá trình đi theo mũi tên. Muốn đến a_j , ở một thời điểm nào đó phải nhảy qua a_i ; nhưng mũi tên nhảy qua a_i phải đi vào một phần tử nhỏ hơn a_i , và phần tử gần nhất như vậy là a_k . Vì thế mũi tên ấy không thể đi đến a_j , vốn nằm bên phải a_k .

1.2.2 Mảnh đất

Xét bảng nhị phân A gồm n hàng và m cột. Cần tìm một hình chữ nhật toàn số 1 có diện tích lớn nhất. Ta gọi đây là bài toán tìm **mảnh đất tối ưu**, theo tên một bài của Olympic Tin học Ba Lan lần thứ 9.

Bài toán có lời giải $O(nm)$. Ký hiệu (i, j) là ô ở hàng thứ i từ trên xuống và cột thứ j từ trái sang. Với mỗi ô, tính

$D[i, j] =$ số ô 1 liên tiếp đi xuống dưới, kể cả ô (i, j) .

Duyệt từ dưới lên: nếu $A[i, j] = 1$ thì $D[i, j] = D[i + 1, j] + 1$, ngược lại $D[i, j] = 0$. Tất cả các giá trị được tính trong $O(nm)$.

Quan sát chủ chốt là mảnh đất tối ưu có thể thu được như sau. Chọn một ô (i, j) , dựng hình chữ nhật có ô ấy trên cạnh trên, chiều cao $D[i, j]$, rồi kéo dài sang trái và phải tối đa cho đến biên bảng hoặc đến một ô 0. Ta sẽ thử mọi cách chọn (i, j) .

Thật vậy, một hình chữ nhật tối ưu có thể được giả sử là cực đại: mỗi cạnh của nó hoặc trùng với biên bảng, hoặc kề một ô 0. Đặc biệt điều này đúng với cạnh dưới. Chọn một ô (i_0, j) ở hàng dưới cùng kề ô 0 (hoặc chọn tùy ý nếu cạnh dưới trùng biên bảng), và gọi i là hàng trên cùng. Hình chữ nhật có chiều cao $D[i, j]$, chứa (i, j) trên cạnh trên, và không kéo dài thêm được về trái hay phải. Nó đúng là một trong các hình mà cách dựng trên xét tới.

Có thể phát biểu quan sát ấy theo cách khác. Với mỗi ô (i, j) có $D[i, j] > 0$, tìm hai chỉ số gần nhất $j' < j < j''$ sao cho

$$D[i, j'] < D[i, j], \quad D[i, j''] < D[i, j],$$

trong đó j' lớn nhất và j'' nhỏ nhất. Hai cột này nằm ngay ngoài hai cạnh dọc của mảnh đất. Quy ước $D[i, 0] = D[i, m + 1] = 0$, diện tích ứng với ô (i, j) là

$$(j'' - j' - 1)D[i, j].$$

Hai chỉ số j' và j'' được tìm bằng đúng thuật toán “tai nạn hàng không”: một lượt từ trái sang phải và một lượt từ phải sang trái trên mỗi hàng của D . Tổng thời gian vẫn là $O(nm)$.

1.2.3 Hình chữ nhật cấp số cộng

Quay lại bảng $n \times m$ các số nguyên không âm. Trước hết xử lý riêng các hình chữ nhật có một cạnh bằng 1 hoặc 2.

Với hình cao 1, mỗi hàng có thể được chia thành các cấp số cộng cực đại, mỗi dãy dài ít nhất hai và hai dãy liên tiếp có đúng một

phần tử chung. Chẳng hạn, dãy $(6, 3, 0, 4, 8)$ được chia thành $(6, 3, 0)$ và $(0, 4, 8)$; dãy $(1, 3, 7, 11, 15)$ được chia thành $(1, 3)$ và $(3, 7, 11, 15)$. Nhờ đó có thể tìm hình cao 1 dài nhất trong $O(nm)$. Làm tương tự cho hình rộng 1, cao 2 và rộng 2.

Từ đây chỉ xét các hình có cả hai cạnh dài ít nhất 3 và quy bài toán về “mảnh đất”. Đánh dấu một ô nếu hình vuông 3×3 nhận ô đó làm tâm là một hình chữ nhật cấp số cộng.

Một hình chữ nhật có hai cạnh ít nhất 3 là hình chữ nhật cấp số cộng khi và chỉ khi mọi ô nằm trong nó, trừ vành biên trong rộng một ô, đều được đánh dấu.

- Chiều thuận là hiển nhiên: nếu hình chữ nhật là cấp số cộng thì mọi hình vuông 3×3 nằm trong nó cũng vậy.
- Ở chiều ngược lại, khi hai ô được đánh dấu kề nhau, các cấp số cộng trong hai hình vuông 3×3 tương ứng ghép lại thành những cấp số cộng dài hơn. Lập luận điem này cho thấy mọi dải cao 3 chạy hết chiều rộng và mọi dải rộng 3 chạy hết chiều cao đều là cấp số cộng. Vì vậy mỗi hàng và mỗi cột của toàn hình đều là một cấp số cộng.

Như vậy, sau khi xử lý riêng các hình “mỏng”, việc tìm hình chữ nhật cấp số cộng đủ dày có diện tích lớn nhất được quy về tìm hình chữ nhật toàn ô đánh dấu.

Còn một chi tiết nhỏ. Trong bài toán mảnh đất, ta chỉ cần hình có diện tích ab lớn nhất. Sau khi thêm lại vành biên rộng một ô, diện tích trở thành $(a+2)(b+2)$. Có thể xảy ra $ab > cd$ nhưng $(a+2)(b+2) < (c+2)(d+2)$, nên mảnh đất có diện tích lớn nhất chưa chắc cho hình chữ nhật cấp số cộng lớn nhất.

Tuy nhiên, hình chữ nhật cấp số cộng tối ưu chắc chắn ứng với một **mảnh đất cực đại**, tức là không thể kéo dài theo bất kỳ hướng nào. Thuật toán của bài toán mảnh đất thực chất đã xét mọi mảnh đất cực đại khi tìm đáp án. Chỉ cần đánh giá diện tích sau khi thêm vành cho từng ứng viên, thay vì chỉ giữ ứng viên có diện tích lõi lớn nhất.

Do đó, toàn bộ bài toán được giải trong thời gian tối ưu $O(nm)$.

Cuộc đua đường phố Bytean



Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Bộ nhớ: 64 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/baj>

Nguồn dịch: bản tiếng Anh, trang in 9–15 (trang PDF vật lý 23–29).

2.1 Đề bài

Ngày mai, Cuộc đua đường phố Bytean sẽ diễn ra tại trung tâm thành phố Bytetown. Các con phố của Bytetown tạo thành một lưới đều: tất cả đều chạy theo hướng nam–bắc hoặc tây–đông. Người tham gia chỉ được phép chạy trên một số đoạn đường đã cho.

Byteasar có nhiệm vụ đặt biểu ngữ của các nhà tài trợ tại một số giao lộ. Để làm việc đó, anh phải nghiên cứu bản đồ đường chạy. Trên bản đồ có n giao lộ và m đoạn đường thẳng đứng hoặc nằm ngang được đánh dấu. Mỗi đoạn bắt đầu và kết thúc tại một giao lộ, không chứa giao lộ nào khác ở bên trong. Các đoạn đường chỉ có thể cắt nhau tại giao lộ.

Các giao lộ được đánh số từ 1 đến n . Cuộc đua bắt đầu tại giao lộ

1 và kết thúc tại giao lộ n . Các vận động viên được tự chọn lộ trình, nhưng chỉ được chạy về phía nam và phía đông, đồng thời chỉ được đi trên các đoạn đã đánh dấu. Những đoạn đường trên bản đồ được chọn sao cho, nếu tuân thủ các quy tắc, từ mọi vị trí đều có thể tới đích và mọi vị trí đều có thể đi tới từ giao lộ xuất phát.

Byteasar muốn đặt các biểu ngữ sao cho không vận động viên nào nhìn thấy biểu ngữ của cùng một nhà tài trợ hai lần. Vì vậy, với một số cặp giao lộ, anh cần kiểm tra liệu có lộ trình của một vận động viên đi qua cả hai giao lộ hay không. Cuộc đua diễn ra ngay ngày mai, nên anh rất cần một chương trình hỗ trợ công việc này.

2.1.1 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, k ($2 \leq n \leq 100\,000$, $1 \leq m \leq 200\,000$, $1 \leq k \leq 300\,000$), lần lượt là số giao lộ trên đường đua, số đoạn đường được đánh dấu và số cặp giao lộ cần kiểm tra.

n dòng tiếp theo mô tả vị trí các giao lộ. Dòng thứ i chứa hai số nguyên x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$), là tọa độ của giao lộ i . Ngoài ra, $x_1 \leq x_n$ và $y_1 \geq y_n$. Không có hai giao lộ nào cùng nằm tại một điểm. Các trục tọa độ tương ứng với các hướng địa lý theo cách tự nhiên: trục OX hướng về phía đông, còn trục OY hướng về phía bắc.

Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên a_i, b_i ($1 \leq a_i, b_i \leq n$, $a_i \neq b_i$), mô tả một đoạn nối hai giao lộ mang số hiệu ấy. Tất cả các đoạn đều thẳng đứng hoặc nằm ngang và chỉ có thể giao nhau tại đầu mút chung, tức các giao lộ.

k dòng sau cùng mô tả các cặp cần kiểm tra. Dòng thứ i chứa hai số nguyên p_i, q_i ($1 \leq p_i, q_i \leq n$, $p_i \neq q_i$).

2.1.2 Dữ liệu ra

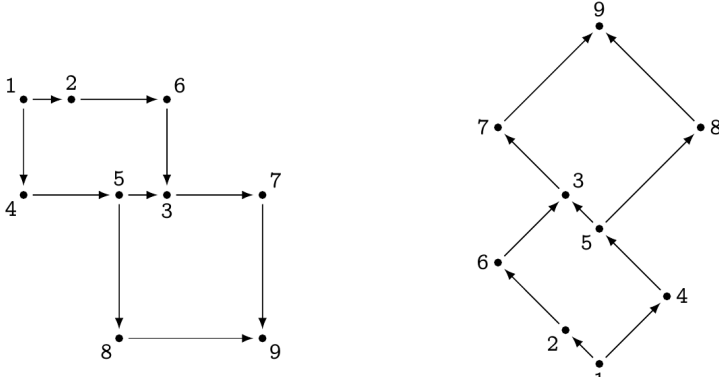
In ra k dòng. Dòng thứ i chứa TAK (nghĩa là “có” trong tiếng Ba Lan) nếu tồn tại lộ trình của một vận động viên đi qua cả hai giao lộ p_i, q_i , theo thứ tự bất kỳ. Nếu không, in NIE (“không”).

2.1.3 Ví dụ

Input	Output
9 10 4	TAK
1 6	NIE
2 6	NIE
4 4	TAK
1 4	
3 4	
4 6	
6 4	
3 1	
6 1	
1 2	
4 1	
2 6	
2 6	
3 6	
5 4	
5 3	
5 8	
3 7	
7 9	
9 8	
4 8	
2 5	
8 7	
7 6	

2.2 Lời giải

Trước hết, hãy quay bản đồ Bytetown 135° sang trái. Khi đó giao lộ xuất phát nằm ở đáy bản đồ, giao lộ đích nằm trên đỉnh, và các đường rời một giao lộ theo hai hướng chéo: lên trái hoặc lên phải. Phép quay không làm thay đổi đáp án, nhưng cách nhìn này thuận tiện hơn.



Hình 2.1: Hình 1. Bản đồ trong ví dụ trước và sau khi quay 135° . Sau phép quay, đỉnh 1 ở dưới cùng và đỉnh 9 ở trên cùng. Nguồn: bản tiếng Anh, trang in 11 (PDF 25).

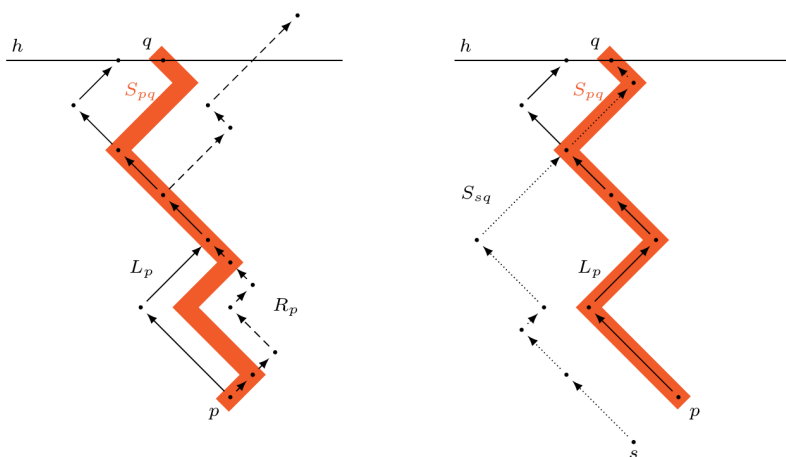
Ta xem bản đồ là một đồ thị có hướng: n đỉnh biểu diễn các giao lộ và m cạnh biểu diễn các đoạn đường có thể chạy qua.

Với một đỉnh v , gọi **đường trái** xuất phát từ v là đường đi bắt đầu tại v , rồi tại mỗi đỉnh tiếp theo luôn rẽ sang trái nếu có thể. Nếu đỉnh chỉ có cạnh sang phải thì đường trái đi theo cạnh ấy. Đường trái kết thúc tại đỉnh đích. **Đường phải** được định nghĩa tương tự. Ký hiệu hai đường này là L_v và R_v . Trong ví dụ ở hình 1, L_4 đi qua 4, 5, 3, 7, 9, còn R_4 đi qua 4, 5, 8, 9.

Giả sử cần kiểm tra có thể đi từ đỉnh $p = (x_p, y_p)$ tới đỉnh $q = (x_q, y_q)$ hay không. Kẻ đường thẳng ngang h qua q . Ta giả sử p nằm dưới h ; nếu không thì hiển nhiên không tồn tại đường từ p tới q , trừ trường hợp $p = q$. Vẽ hai đường L_p, R_p .

Điều kiện 1. Đường L_p cắt h tại q hoặc về bên trái q , đồng thời R_p cắt h tại q hoặc về bên phải q .

Nếu có đường S_{pq} từ p tới q , điều kiện này nhất thiết đúng: S_{pq} cắt h tại q ; đường trái từ p không bao giờ có thể nằm về bên phải S_{pq} , và tương tự, đường phải từ p không bao giờ nằm về bên trái S_{pq} .



Hình 2.2: Hình 2. Bên trái: đường S_{pq} nằm trong miền kẹp giữa L_p và R_p . Bên phải: S_{sq} cắt L_p , nhờ đó ghép được một đường từ p đến q . Nguồn: bản tiếng Anh, trang in 12 (PDF 26).

Chiều ngược lại cũng đúng. Giả sử điều kiện 1 được thỏa mãn. Xét một đường S_{sq} từ đỉnh xuất phát s tới q ; đường này chắc chắn tồn tại vì từ s có thể tới mọi đỉnh. Mọi đường tới q bắt đầu tại p hoặc ở phía dưới p đều phải cắt L_p hoặc R_p . Giả sử S_{sq} cắt L_p . Ta đi từ p theo L_p tới giao điểm, rồi chuyển sang S_{sq} và đi tiếp tới q . Trường hợp cắt R_p hoàn toàn tương tự. Vậy để kiểm tra khả năng đi từ p tới q , chỉ cần kiểm tra điều kiện 1.

2.2.1 Cấu trúc của các đường trái và phải

Các đường trái và phải có cấu trúc rất chặt chẽ. Muốn dựng đường trái từ v , ta đi theo cạnh đầu tiên — sang trái nếu có, nếu không thì sang phải — để tới w . Từ đó trở đi, phần còn lại của L_v trùng hoàn toàn với L_w .

Ta dựng mảng hai chiều `left` để di chuyển nhanh trên các đường trái bằng kỹ thuật **binary lifting**. Với mọi đỉnh v và $0 \leq i \leq \lfloor \log_2 n \rfloor$, `left[v][i]` là đỉnh đạt tới sau 2^i bước trên L_v . Mảng `right` cho các đường phải được dựng tương tự. Để tiện xử lý, thêm tại đỉnh đích một khuyên đi từ chính nó về nó; nhờ vậy mọi ô trong hai mảng đều được xác định.

Giá trị `left[v][0]` chính là đỉnh ở cuối cạnh đầu tiên của L_v . Sau đó tính lần lượt theo công thức

$$\text{left}[v][i] = \text{left}[\text{left}[v][i-1]][i-1].$$

Thật vậy, trước hết ta đi 2^{i-1} bước từ v để tới w , sau đó đi thêm 2^{i-1} bước trên L_w , vốn trùng với phần còn lại của L_v . Hai mảng `left` và `right` đều có $O(n \log n)$ ô, nên được dựng trong tổng thời gian $O(n \log n)$.

Sau khi có hai mảng, ta kiểm tra điều kiện 1 bằng một dạng tìm kiếm nhị phân. Chẳng hạn, cần tìm nơi L_p cắt đường h . Bắt đầu ở p , ta đi dọc L_p nhưng luôn bảo đảm chưa vượt lên trên h . Với $i = \lfloor \log_2 n \rfloor, \dots, 0$, xét đỉnh đạt được sau 2^i bước từ đỉnh hiện tại. Nếu đỉnh đó cao hơn h thì không đi; nếu nó nằm trên chính h hoặc ở dưới h thì thực hiện bước nhảy 2^i .

Kết thúc quá trình, ta ở đỉnh cao nhất w trên L_p mà không cao hơn h . Chỉ còn xác định cạnh đầu tiên của L_w cắt h về phía nào so với q . Làm tương tự với R_p là kiểm tra được điều kiện 1.

Như vậy, sau $O(n \log n)$ thời gian tiền xử lý, mỗi truy vấn được trả lời trong $O(\log n)$.

2.2.2 Lời giải nhanh hơn

Mặc dù lời giải trên đã đủ nhanh, ta còn có một thuật toán thú vị với thời gian tiền xử lý tuyến tính. Trước hết xét một bài toán tưởng như không liên quan.

Vị trí tương đối của hai đỉnh trong cây

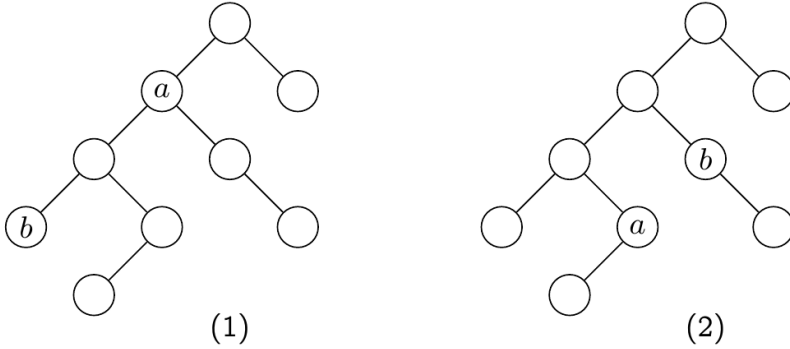
Xét một cây nhị phân có gốc, trong đó mỗi đỉnh có không, một hoặc hai con. Nếu có hai con, ta phân biệt con trái và con phải. Chọn hai đỉnh khác nhau a, b , và gọi S_a, S_b là các đường từ gốc tới chúng.

Chỉ có đúng bốn khả năng: a nằm **trên** b (tức $a \in S_b$); a nằm **dưới** b (tức $b \in S_a$); a nằm **bên trái** b (khi đi từ a về gốc, ta nhập vào S_b từ bên trái); hoặc a nằm **bên phải** b . Hai trường hợp đầu đối xứng với nhau, hai trường hợp sau cũng vậy.

Chạy DFS trên cây. Với mỗi đỉnh v , ghi thời điểm vào I_v và thời điểm ra O_v , tức thời điểm thăm v lần đầu và thời điểm ngay sau khi đã duyệt xong toàn bộ cây con của v . Vị trí tương đối của hai đỉnh phân biệt a, b được xác định như sau:

- a nằm trên b khi và chỉ khi $I_a < I_b$ và $O_b < O_a$;
- a nằm dưới b khi và chỉ khi $I_b < I_a$ và $O_a < O_b$;
- a nằm bên trái b khi và chỉ khi $O_a < I_b$;
- a nằm bên phải b khi và chỉ khi $O_b < I_a$.

Vì thế, sau $O(n)$ thời gian tiền xử lý, ta xác định được vị trí tương đối của hai đỉnh trong $O(1)$.



Hình 2.3: Hình 3. Hai trường hợp tiêu biểu: a nằm trên b , và a nằm bên trái b . Hai trường hợp còn lại thu được bằng cách đổi vai a, b .
 Nguồn: bản tiếng Anh, trang in 13 (PDF 27).

Lời giải thời gian tuyến tính

Ta dựng đồ thị của các đường trái bằng cách lấy hợp của đường trái xuất phát từ mọi đỉnh. Đồ thị này có cùng n đỉnh với bản đồ Bytetown, nhưng từ mỗi đỉnh, trừ đỉnh đích, chỉ giữ đúng một cạnh: cạnh trái nếu có, nếu không thì cạnh phải.

Đồ thị thu được là một cây nhị phân có gốc tại đỉnh đích. Đây cũng là lý do ta quay bản đồ để đích ở trên: cây đường trái có gốc ở trên và mọc xuống dưới như cách thường vẽ cây. Gọi cây này là T_L . Tương tự, dựng cây đường phải T_R .

Một lần nữa, cần kiểm tra có đường từ p tới q hay không. Kẻ đường ngang h qua q và xét vị trí tương đối của p, q trong T_L :

- Nếu p nằm dưới q , đường cần tìm tồn tại ngay trong T_L .
- p không thể nằm trên q nếu về mặt hình học p ở dưới h , vì khi ấy p phải nằm phía trên h .
- Nếu p nằm bên trái q , hai đường L_p, L_q nhập lại tại một đỉnh, trong đó L_p đi vào từ trái và L_q đi vào từ phải. Lăn ngược

đồng thời hai đường xuống tới h , L_q gặp q , còn L_p gặp h bên trái q .

- Nếu p nằm bên phải q , L_p cắt h bên phải q .

Đây chính là một nửa của điều kiện 1. Do đó điều kiện về đường trái tương đương với: trong T_L , p nằm **dưới** hoặc **bên trái** q . Tương tự, điều kiện về đường phải tương đương với: trong T_R , p nằm **dưới** hoặc **bên phải** q .

Tóm lại, trước tiên dựng T_L, T_R , rồi chạy DFS trên mỗi cây để lấy thời điểm vào/ra. Cả hai bước tốn $O(n)$. Với mỗi cặp không có thứ tự trong đề, chọn đỉnh thấp hơn trên bản đồ đã quay làm p , đỉnh cao hơn làm q . Có thể so sánh độ cao này ngay trên tọa độ ban đầu bằng $x - y$, vì mỗi bước chạy về đông hoặc về nam đều làm đại lượng ấy tăng. Nếu hai đỉnh khác nhau có cùng độ cao thì chúng không thể cùng nằm trên một lộ trình. Sau đó đáp án là TAK khi và chỉ khi đồng thời:

1. p không nằm cao hơn q ;
2. trong T_L , p nằm dưới hoặc bên trái q ;
3. trong T_R , p nằm dưới hoặc bên phải q .

Mọi phép kiểm tra đều là $O(1)$. Tổng thời gian là $O(n + m + k)$ nếu tính cả việc đọc và định hướng m đoạn đường; phần tiền xử lý sau khi đã có đồ thị là $O(n)$, mỗi truy vấn $O(1)$, và bộ nhớ sử dụng là $O(n + m)$.

Kết quả tối ưu này có được nhờ cấu trúc đặc biệt của đồ thị: phẳng, không chu trình, chỉ có đúng một nguồn và một đích. Với đồ thị tổng quát, bài toán khó hơn nhiều: thuật toán được biết đến trong tài liệu nguồn để trả lời truy vấn trong $O(1)$ cần $O(n^{2.38})$ thời gian tiền xử lý và sử dụng phép nhân ma trận nhanh.

Liệu nó có dừng?

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 64 MB

Mã bài: czy

Nguồn: AMPPZ 2011¹

Bản dịch dựa trên bản tiếng Anh, các trang in 16–18
(trang PDF vật lý 30–32), của *Looking for a Challenge 2*.

3.1 Đề bài

Bajtazar đang đi dạo quanh thư viện của Đại học Warszawa thì nhìn thấy trên một mặt tiền của thư viện một đoạn chương trình kèm dòng chữ “Liệu nó có dừng?”. Câu hỏi có vẻ thú vị, nên sau khi về nhà, Bajtazar đã thử giải đáp nó. Không may, khi chép lại đoạn mã, cậu đã ghi nhầm thành:

```
while n > 1 do
  if n mod 2 = 0 then
    n := n / 2
  else
    n := 3 · n + 3
```

¹<https://oi.edu.pl/en/archive/amppz/2011/czy>

Bajtazar muốn xác định những giá trị ban đầu nào của biến n làm chương trình mà cậu đã chép dừng lại. Ta giả sử biến n có kích thước không giới hạn, nghĩa là nó có thể nhận các giá trị lớn tùy ý.

3.1.1 Dữ liệu vào

Dòng đầu tiên và duy nhất chứa một số nguyên n ($2 \leq n \leq 10^{14}$). Hãy kiểm tra chương trình đã cho có dừng với giá trị n này hay không.

3.1.2 Kết quả

Trên dòng đầu tiên và duy nhất, in một từ TAK (nghĩa là “có” trong tiếng Ba Lan) nếu chương trình dừng với giá trị n đã cho; ngược lại, in NIE (nghĩa là “không”).

3.1.3 Ví dụ

Dữ liệu vào

4

Kết quả

TAK

3.2 Lời giải

Trong đề bài có một đoạn chương trình mà ta cần phân tích hành vi. Vì đã biết mã nguồn, ta có thể chép nguyên đoạn mã đó vào lời giải rồi chạy. Nếu với một giá trị n cho trước mà chương trình dừng, ta lập tức biết câu trả lời. Trường hợp ngược lại khó hơn một chút: làm sao có thể chắc chắn rằng đoạn mã đang chạy sẽ không bao giờ dừng?

Ta có thể bắt đầu bằng một giả định rất táo bạo — và không hẳn có căn cứ: nếu đoạn mã chưa dừng sau, chẳng hạn, 100 000 vòng lặp thì ta coi như nó sẽ không bao giờ dừng.

Cách làm ấy rất dễ cài đặt, nhưng có hai nhược điểm. Thứ nhất, dựa vào đâu ta biết 100 000 vòng lặp là đủ? Thứ hai, nếu biểu diễn n bằng số nguyên có dấu 64 bit (chẳng hạn kiểu long long trong C++), ta sẽ rơi vào một cái bẫy. Dù giá trị ban đầu của n theo đề bài không vượt quá 10^{14} , sau một số vòng lặp nó có thể trở nên lớn hơn rất nhiều. Ví dụ, với $n = 366\,713\,142\,269$, sau khoảng 160 vòng lặp, n đạt

$$10\,010\,331\,589\,553\,303\,736 > 10^{19},$$

trong khi số nguyên có dấu 64 bit chỉ biểu diễn được các số không lớn hơn

$$2^{63} - 1 \approx 9,2234 \cdot 10^{18}.$$

Hãy tiếp cận bài toán theo cách khác. Ta thử quan sát hành vi của đoạn mã với mọi n từ 1 đến 20; máy tính có thể giúp làm việc này. Rất nhanh chóng, ta nhận thấy chương trình dừng với n bằng 1, 2, 4, 8, 16. Với những giá trị còn lại, nó lặp vô hạn vì kể từ một thời điểm nào đó, n lần lượt nhận các giá trị

$$3, 12, 6, 3, 12, 6, 3, 12, 6, \dots$$

Có vẻ như chương trình dừng đúng với các giá trị là lũy thừa của 2. Thật vậy, nếu n là một lũy thừa của 2, đoạn mã sẽ liên tục chia n cho 2 cho đến khi $n = 1$. Do đó, trong trường hợp này chương trình chắc chắn dừng.

Ta có thể đoán rằng với mọi giá trị n còn lại, chương trình sẽ chạy mãi mãi. Phỏng đoán này đúng, nhưng vì sao?

Nếu giá trị ban đầu của n là số lẻ, phép gán

$$n := 3n + 3$$

sẽ được thực hiện. Dễ thấy giá trị mới của n chia hết cho 3. Hơn nữa, từ thời điểm đó trở đi, n luôn chia hết cho 3. Quả vậy, nếu n chia hết cho 3 thì bất kể ta thực hiện phép gán $n := n/2$ hay

$n := 3n + 3$, giá trị mới vẫn chia hết cho 3. Vì vậy, ta không bao giờ có thể đạt tới $n = 1$.

Chỉ còn trường hợp n chẵn nhưng không phải lũy thừa của 2. Khi đó, chương trình sẽ liên tục chia n cho 2 cho đến khi nó trở thành số lẻ. Số lẻ này phải lớn hơn 1, bởi chỉ khi giá trị ban đầu là một lũy thừa của 2 thì quá trình chia liên tiếp mới kết thúc ở 1. Một khi n đã trở thành số lẻ lớn hơn 1, theo phân tích ở trên, chương trình sẽ không dừng.

Vậy toàn bộ bài toán chỉ còn là kiểm tra n có phải lũy thừa của 2 hay không. Có thể kiểm tra điều kiện này rất dễ dàng: chừng nào n còn chia hết cho 2, ta tiếp tục chia nó cho 2. Nếu cuối cùng thu được $n = 1$, giá trị ban đầu là một lũy thừa của 2; ngược lại thì không phải.

3.2.1 Chứng minh tính đúng đắn

Ta chứng minh thuật toán in TAK khi và chỉ khi chương trình dừng.

- Nếu n là lũy thừa của 2, viết $n = 2^k$. Mỗi vòng lặp chia n cho 2, nên sau đúng k lần chia ta thu được $n = 1$ và vòng lặp dừng. Thuật toán cũng chia hết các thừa số 2 và nhận được 1, do đó in TAK.
- Nếu n không phải lũy thừa của 2, sau khi chia hết các thừa số 2, ta thu được một số lẻ lớn hơn 1. Ở vòng tiếp theo, giá trị mới là $3n + 3$, nên chia hết cho 3. Từ đó về sau, cả hai phép biến đổi $n := n/2$ (chỉ được thực hiện khi n chẵn) và $n := 3n + 3$ đều bảo toàn tính chia hết cho 3. Vì 1 không chia hết cho 3, chương trình không thể đạt tới 1 và do đó không dừng. Thuật toán nhận được một số khác 1 nên in NIE.

Vậy thuật toán luôn cho câu trả lời đúng.

3.2.2 Độ phức tạp

Thuật toán thực hiện nhiều nhất $\lfloor \log_2 n \rfloor$ phép chia, nên có độ phức tạp thời gian $O(\log n)$ và dùng $O(1)$ bộ nhớ.

3.2.3 Một phép kiểm tra bằng toán tử bit

Ta còn có thể kiểm tra nhanh hơn bằng các toán tử trên bit. Toán tử AND theo bit (ký hiệu & trong C++) cho kết quả có bit 1 đúng tại những vị trí mà cả hai toán hạng đều có bit 1. Toán tử XOR theo bit (ký hiệu ^) cho kết quả có bit 1 tại những vị trí mà hai toán hạng khác nhau.

Để kiểm tra n có phải lũy thừa của 2 hay không, chỉ cần kiểm tra

$$((n \text{ xor } (n - 1)) \text{ and } n) = n.$$

Một dạng tương đương và quen thuộc hơn, với $n > 0$, là

$$(n \text{ and } (n - 1)) = 0.$$

3.2.4 Bài toán Collatz

Trên mặt tiền thư viện của Đại học Warszawa thực sự có một đoạn mã gần giống đoạn mã trong đề. Tuy nhiên, đề bài nói rằng Bajtazar đã chép nhầm. Đoạn mã gốc trên tường thư viện là:

```
while n > 1 do
  if n mod 2 = 0 then
    n := n / 2
  else
    n := 3 · n + 1
```

Ở dòng cuối là $n := 3n + 1$ thay vì $n := 3n + 3$. Dù hai đoạn mã chỉ khác nhau rất ít, khác biệt ấy lại có ý nghĩa lớn. Trong bài toán này, ta đã mô tả được tất cả các số làm chương trình dừng. Bài toán tương tự đối với đoạn mã trên tường thư viện khó hơn rất nhiều.

Năm 1937, nhà toán học người Đức Lothar Collatz đưa ra giả thuyết rằng đoạn mã trên dừng với mọi số nguyên dương n . Nhiều nhà toán học đã nghiên cứu giả thuyết này, nhưng tại thời điểm cuốn sách được biên soạn vẫn chưa ai chứng minh hoặc bác bỏ được nó. Bản tiếng Anh cho biết giả thuyết khi ấy đã được kiểm chứng bằng máy tính với mọi

$$n < 20 \cdot 2^{58} \approx 5,7646 \cdot 10^{18},$$

và thường được tin là đúng, nhưng chưa có một chứng minh hình thức. Vì vậy, nếu đề bài dùng đúng phép biến đổi của bài toán Collatz nguyên bản, câu hỏi “Liệu nó có dừng?” sẽ chưa thể được trả lời chắc chắn trong mọi trường hợp.

Cây và đàn kiến ★★★★★

Tác giả đề: Szymon Acedański

Tác giả lời giải: Szymon Acedański, Tomasz Idziaszek

Bộ nhớ: 6 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/drz>

Nguồn dịch: đối chiếu bản tiếng Anh *Ants* và bản tiếng Ba Lan *Drzewo i mrówki*, trang in 19–22 (bản tiếng Anh: trang PDF vật lý 33–36).

4.1 Đề bài

Những người làm tin học thích cây. Kiến cũng thích cây. Vì vậy, ta được cho một cái cây có hai con kiến bò trên đó — Kiến Trái và Kiến Phải — theo cách thể hiện trong hình minh họa của đề gốc (chúng bò theo đường chấm). Chúng bắt đầu hành trình ở đầu dưới của thân cây, mỗi con ở một phía đối diện.

Khi đi xa dần khỏi gốc (đi lên), Kiến Trái mất 2 giây để đi hết một cạnh; khi đi về phía gốc (đi xuống), nó mất 1 giây. Kiến Phải nhanh gấp đôi Kiến Trái. Mỗi khi gặp nhau, cả hai con đều quay đầu và bắt đầu bò theo chiều ngược lại. Nếu một con bò từ cây xuống mặt đất, nó lập tức bắt đầu trèo lên phía bên kia của thân cây.

Ngoài ra, hai con kiến nhỏ đến mức ngay cả kính hiển vi cũng

không nhìn thấy chúng (trong hình, chúng được cố ý vẽ lớn hơn rất nhiều). Hãy viết chương trình tính thời điểm hai con kiến quay đầu lần thứ hai.

4.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên t ($1 \leq t \leq 1000$), là số bộ dữ liệu.

Mỗi bộ dữ liệu gồm hai dòng. Dòng đầu chứa số nguyên chẵn n ($2 \leq n \leq 100\,000\,000$), là số cạnh của cây. Dòng thứ hai chứa mô tả của cây: một xâu dài $n/2$, biểu diễn một số nhị phân $2n$ bit dưới dạng hệ thập lục phân, dùng các chữ số và các chữ cái thường từ a đến f.

Số này mô tả đường đi vòng quanh toàn bộ cây của Kiến Trái, giả sử Kiến Phải đứng yên. Đọc các bit từ trái sang phải, mỗi bit cho biết trên cạnh tiếp theo của hành trình, Kiến Trái đang đi xa khỏi gốc (bit 1) hay đi về phía gốc (bit 0). Cây có một thân, nghĩa là từ gốc chỉ có đúng một cạnh đi ra.

Kích thước tệp dữ liệu vào không vượt quá 50 MB. Lưu ý rằng con số này lớn hơn đáng kể lượng bộ nhớ dành cho chương trình.

4.1.2 Dữ liệu ra

In ra t dòng, lần lượt là đáp án cho các bộ dữ liệu. Mỗi đáp án là thời điểm (tính bằng giây) hai con kiến quay đầu lần thứ hai, viết dưới dạng phân số tối giản p/q (không có khoảng trắng quanh dấu /), trong đó p, q là các số nguyên dương. Nếu đáp án là số nguyên thì, hiển nhiên, $q = 1$.

4.1.3 Ví dụ

Input	Output
1	282/5
28	
fb1da30d1b7230	

Bộ dữ liệu mẫu ứng với hình trong đề gốc và được chuyển thành dãy bit sau:

1111 1011 0001 1101 1010 0011 0000 1101 0001 1011 0111 0

4.2 Lời giải

Giới hạn dữ liệu được chọn sao cho không thể lưu toàn bộ mô tả cây trong bộ nhớ. Lời giải chuẩn chỉ đọc mô tả đúng một lần và xử lý trực tuyến, không lưu lại. Phác thảo thuật toán như sau:

- Đọc kích thước cây; nhờ đó biết tổng số đoạn đi “lên” và đi “xuống”.
- Trong khi đọc mô tả, mô phỏng hành trình của Kiến Trái cho đến lần gặp thứ nhất. Trên mỗi cạnh, ta có thể kiểm tra cuộc gặp có xảy ra tại đó hay không: từ quãng đường Kiến Trái đã đi, ta suy ra được quãng đường Kiến Phải phải đi để tới điểm gặp.
- Sau cuộc gặp, cả hai con đi về phía gốc. Tiếp tục đọc dữ liệu, lần này mô phỏng hành trình của Kiến Phải. Điều có vẻ bất ngờ là Kiến Trái tới gốc trước, mặc dù Kiến Phải chạy nhanh hơn. Ta sẽ giải thích điều này ở dưới.
- Sau khi Kiến Trái tới gốc, tiếp tục mô phỏng Kiến Phải cho đến điểm gặp thứ hai.

Sau đây là mô tả chính thức hơn. Một điểm trên cây có thể được biểu diễn bởi cặp số thực (a, k) , trong đó:

- (a) là tổng số cạnh Kiến Trái phải đi qua từ gốc để tới điểm ấy; nếu một cạnh được đi qua theo cả hai chiều thì nó được tính hai lần;
- (k) là độ cao của điểm, tức số cạnh trên đường từ gốc đến điểm ấy.

Nếu Kiến Trái mất $t_{\uparrow}$ giây cho một cạnh đi lên và $t_{\downarrow}$ giây cho một cạnh đi xuống, thời gian để nó tới (a, k) là

$$[a, k, t_{\uparrow}, t_{\downarrow}] := \frac{a+k}{2}t_{\uparrow} + \frac{a-k}{2}t_{\downarrow}.$$

Vì thế, điểm gặp thứ nhất (a_1, k_1) thỏa

$$[a_1, k_1, 2, 1] = [2n - a_1, k_1, 1, \frac{1}{2}]. \quad (1)$$

Ta tìm điểm này bằng cách mô phỏng Kiến Trái trong lúc đọc từng cạnh. Giả sử Kiến Trái đang ở (a, k) , đầu mút của cạnh sắp xét. Đặt $b = 1$ nếu cạnh này đi lên và $b = -1$ nếu nó đi xuống. Nếu cuộc gặp xảy ra sau khi đi một đoạn dài ε trên cạnh thì

$$a_1 = a + \varepsilon, \quad k_1 = k + b\varepsilon.$$

Thế vào (1), ta được

$$\varepsilon = \frac{6n - 9a - k}{9 + b}.$$

Nếu $0 \leq \varepsilon < 1$, ta đã xác định được điểm gặp thứ nhất. Thời gian đi tới đó là

$$t_1 = [a_1, k_1, 2, 1] = \frac{3a_1 + k_1}{2}.$$

Giả sử sau cuộc gặp, ta để cả hai con tiếp tục quay về gốc. Kiến Trái sẽ tới gốc sau

$$[a_1, -k_1, 2, 1] = t_1 - k_1$$

giây tính từ lúc xuất phát, còn Kiến Phải sẽ tới gốc sau

$$[2n - a_1, -k_1, 1, \frac{1}{2}] = t_1 - \frac{k_1}{2}.$$

Do đó Kiến Trái luôn tới gốc trước Kiến Phải. Từ đây ta bắt đầu tìm điểm gặp thứ hai bằng cách mô phỏng hành trình của Kiến Phải.

Giả sử Kiến Phải đi một đoạn dài ε' trên cạnh bắt đầu tại (a_0, k_0) , và tới điểm gặp thứ hai (a_2, k_2) . Đặt $b' = 1$ nếu cạnh đang xét đi lên, $b' = -1$ nếu nó đi xuống. Khi đó

$$a_2 = a_0 + \varepsilon', \quad k_2 = k_0 + b'\varepsilon',$$

và điều kiện đồng bộ thời gian là

$$[a_1, -k_1, 2, 1] + [2n - a_2, k_2, 2, 1] = [a_2 - a_1, k_2 - k_1, 1, \frac{1}{2}].$$

Suy ra

$$\varepsilon' = \frac{12n - 9(a_0 - a_1) + (k_0 - k_1)}{9 - b'}.$$

Nếu $0 \leq \varepsilon' < 1$, ta đã tìm được điểm gặp thứ hai. Khoảng thời gian từ lần gặp thứ nhất đến điểm này là

$$t_2 = [a_2 - a_1, k_2 - k_1, 1, \frac{1}{2}] = \frac{3(a_2 - a_1) + (k_2 - k_1)}{4}.$$

Vì vậy, tổng thời gian từ lúc xuất phát đến lần gặp thứ hai là

$$t_1 + t_2 = \frac{3(a_1 + a_2) + k_1 + k_2}{4}.$$

Ta còn phải xử lý các phân số mà không dùng số thực. Giá trị ε là bội của $1/(9 + b)$; biến đổi đại số tương tự cho thấy ε' là bội của

$$\frac{9 - b}{(9 + b)(9 - b')}.$$

Vì $b, b' \in \{-1, 1\}$, suy ra a_1, a_2, k_1, k_2 đều là bội của $1/800$. Do đó, mọi phép tính có thể được thực hiện chính xác bằng số nguyên 64 bit: chỉ cần lưu các đại lượng sau khi nhân với 800.

Mỗi bước của thuật toán đọc đúng một cạnh kế tiếp và thực hiện một số hàng phép toán. Vì vậy, độ phức tạp thời gian là $O(n)$, còn bộ nhớ phụ là $O(1)$.

Thông tin nguồn

- *Looking for a Challenge 2*, bản tiếng Anh, Warszawa, 2019.
- *W poszukiwaniu wyzwania 2*, bản tiếng Ba Lan, ấn bản sửa đổi lần hai, phiên bản 2.1, Warszawa, 2019.